

ВВЕДЕНИЕ

Актуальность темы обусловлена необходимостью создания надёжных и масштабируемых решений для мониторинга физических величин в реальном времени. Традиционные проводные системы ограничены в развитии, тогда как беспроводные *IoT*-платформы в связке с лёгкими протоколами (*MQTT*) позволяют разрабатывать компактные, энергоэффективные и легко интегрируемые системы телеметрии. Контроль температуры и автоматическое управление охлаждением являются базовыми задачами для серверной инфраструктуры, промышленного оборудования и бытовой электроники, что делает разработку подобных распределённых систем практически значимой.

Целью работы является разработка и апробация распределённой системы телеметрии для контроля температуры и управления работой кулера на базе микроконтроллера *ESP32* и серверного приложения, реализованного на языке *Rust*.

Для достижения цели поставлены следующие задачи:

- провести анализ аппаратных платформ и протоколов обмена данными, обосновать выбор стека технологий;
- разработать алгоритмическое и программное обеспечение для микроконтроллера (сбор данных, логика управления, передача по сети);
- реализовать серверную часть системы с функциями приёма, парсинга *JSON*, визуализации и расчёта статистики в реальном времени;
- выполнить интеграционное тестирование и верификацию работоспособности всех подсистем.

Объектом исследования стали распределённые системы мониторинга и управления в реальном времени.

В качестве предмета исследования выступает программно-аппаратная реализация телеметрического канала передачи данных температуры и алгоритмов управления исполнительным устройством.

Практическая значимость работы заключается в создании работоспособного прототипа, готового к адаптации под реальные условия эксплуатации. Применение событийно-ориентированной архитектуры и языка *Rust* обеспечивает высокую производительность серверной части, безопасность памяти и минимальное потребление ресурсов, что критически важно для систем круглосуточного мониторинга. Полученные результаты могут служить базовым модулем при проектировании более сложных распределённых *IoT*-решений и образовательным материалом по дисциплинам, связанным с сетевыми и распределёнными информационными системами.

1 ОБЗОР ПРОГРАММНО-ТЕХНИЧЕСКОЙ БАЗЫ СИСТЕМЫ ТЕЛЕМЕТРИИ

1.1 Информационно-логическая модель предметной области

Информационно-логическая модель предметной области представляет собой формализованное описание структуры данных, информационных потоков, состояний компонентов и правил их взаимодействия в рамках распределённой системы мониторинга и управления. Модель определяет функциональные границы системы, выделяет ключевые сущности, устанавливает роли участников обмена данными и описывает механизмы синхронизации состояний в условиях асинхронной сетевой среды. Предметная область охватывает процессы непрерывного контроля температурного режима и автоматического управления исполнительным устройством (вентилятором охлаждения) с обеспечением минимальной задержки реакции, устойчивости к разрывам связи и возможности масштабирования количества измерительных узлов.

Ключевыми сущностями модели являются: источник аналогового сигнала (*NTC*-термистор), краевое устройство сбора и первичной обработки (микроконтроллер *ESP32*), транспортный слой (локальная сеть *Wi-Fi*, протокол *MQTT*, брокер сообщений *Mosquitto*), серверный узел агрегации и визуализации (приложение на языке *Rust*), а также исполнительный механизм (вентилятор постоянного тока). Взаимодействие между сущностями строится на принципах слабой связанности (*loose coupling*), асинхронного обмена сообщениями и разделения контуров измерения и управления, что характерно для современных IoT-архитектур.

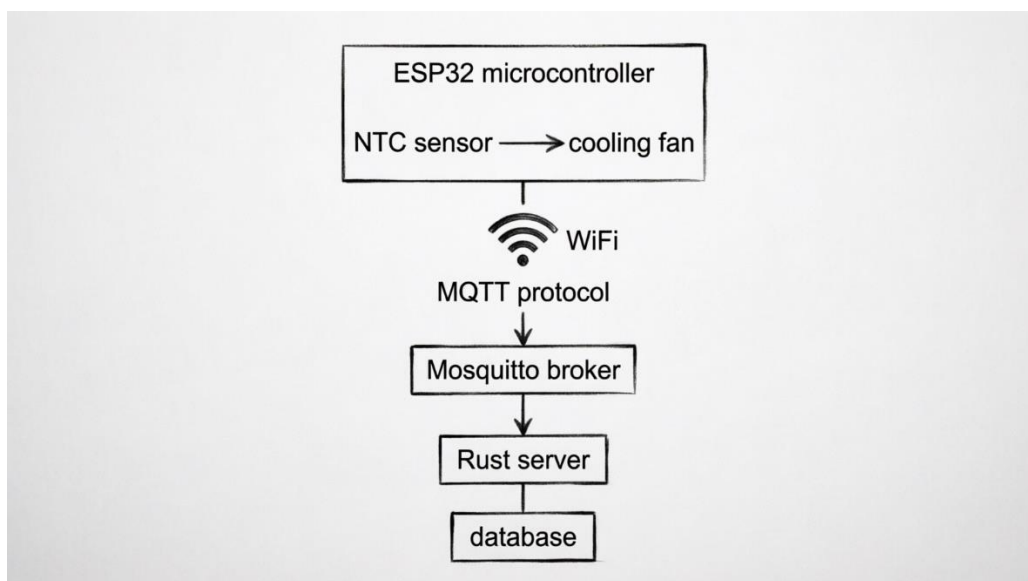


Рисунок 1.1 – Архитектура распределённой системы телеметрии температуры

Информационный поток строится по событийно-ориентированной архитектуре с использованием паттерна «издатель-подписчик» (*Publish/Subscribe*). Система декомпозируется на три логических уровня, что обеспечивает модульность и отказоустойчивость.

Жизненный цикл телеметрического сообщения в рамках модели включает следующие этапы:

- генерация данных: микроконтроллер инициирует опрос аналого-цифрового преобразователя (АЦП), получает сырое значение сопротивления термистора и применяет калибровочную зависимость (формула Стейнхарта-Харта) для преобразования в температурное значение в градусах Цельсия.

- упаковка и сериализация: полученные метрики агрегируются в структурированный *JSON*-объект, содержащий идентификатор устройства, текущую температуру, состояние управляющего выхода, пороговое значение срабатывания и временную метку аптайма. Сериализация выполняется на стороне микроконтроллера с минимальными накладными расходами.

- публикация и маршрутизация: пакет передаётся в *MQTT*-топик *telemetry/temperature* с уровнем качества обслуживания *QoS 0* (доставка не гарантируется, но обеспечивается минимальная задержка и нагрузка на сеть). Брокер *Mosquitto* принимает сообщение, сопоставляет его с подписками активных клиентов и выполняет асинхронную рассылку.

- приём и обработка: серверное приложение, выступающее подписчиком, получает поток сообщений, выполняет десериализацию *JSON*, валидацию типов данных и диапазонов значений. Корректные пакеты передаются в модуль агрегации, где вычисляются статистические показатели (минимум, максимум, скользящее среднее, количество записей).

- визуализация и логирование: обработанные данные выводятся в консольный интерфейс мониторинга в реальном времени с привязкой к системному времени сервера. Архитектура предусматривает возможность последующего расширения модели за счёт подключения модулей долговременного хранения (*SQLite/PostgreSQL*) или веб-интерфейса.

Особенностью логической модели является разделение контура измерения и контура управления. Логика принятия решений о включении/выключении вентилятора реализована на уровне микроконтроллера с применением гистерезиса (порог срабатывания 35°C, порог отключения 33°C), что исключает эффект «дребезга» состояния и обеспечивает стабильную работу исполнительного механизма. Состояние вентилятора синхронизируется с сервером путём включения флага *fan_state* в телеметрический пакет, что позволяет оператору или автоматизированным системам верхнего уровня отслеживать фактический режим работы без необходимости отправки управляющих команд в обратном направлении.

Модель также предусматривает механизмы обработки исключительных ситуаций и обеспечения надёжности распределённого взаимодействия. При

потере сетевого соединения краевое устройство сохраняет последнее известное состояние, продолжает выполнять локальную логику управления и автоматически восстанавливает *MQTT*-сессию после стабилизации канала связи. На серверной стороне реализована защита от некорректных пакетов: сообщения с нарушенной структурой *JSON* или выходом температурных значений за физически допустимые границы отбрасываются с логированием ошибки, что предотвращает падение приложения и искажение статистики. Использование асинхронной модели обработки событий в *Rust (tokio + rumqtte)* гарантирует неблокирующий приём потока данных даже при высокой частоте публикаций от нескольких узлов телеметрии.

Так информационно-логическая модель обеспечивает чёткое разделение ответственности между подсистемами, детерминированную обработку телеметрических данных, устойчивость к сетевым сбоям и готовность к интеграции дополнительных измерительных узлов без модификации серверной логики, что полностью соответствует требованиям к распределённым информационным системам промышленного и лабораторного класса.

1.2 Сравнение аппаратных платформ

Выбор аппаратной платформы для краевого узла распределённой системы телеметрии является определяющим этапом проектирования, поскольку от технических характеристик микроконтроллера напрямую зависят точность измерений, стабильность сетевого взаимодействия, ресурсоёмкость программного стека, энергоэффективность и общая надёжность системы. В условиях распределённой архитектуры, где устройство функционирует автономно в локальной сети *Wi-Fi*, выполняет преобразование аналоговых сигналов в цифровую форму, реализует логику гистерезисного управления исполнительным механизмом и транслирует структурированные данные по протоколу *MQTT*, к платформе предъявляется комплекс требований. К ним относятся: достаточная вычислительная мощность для обработки данных в реальном времени, объём оперативной памяти, сопоставимый с требованиями сетевого стека *TCP/IP* и библиотек сериализации *JSON*, наличие встроенных средств беспроводной связи, высокая разрядность и калибруемость аналого-цифрового преобразователя (АЦП), развитая программная экосистема, а также доступность и стоимость компонентов.

Для обоснованного выбора были проанализированы четыре наиболее распространённые в образовательной и промышленной практике микроконтроллерные платформы: *Arduino Uno* на базе *ATmega328P*, *STM32F103C8T6* («*Blue Pill*»), *Raspberry Pi Pico* на базе *RP2040* и *Espressif ESP32*. Платформа *Arduino Uno* исторически стала стандартом де-факто для начального изучения микроконтроллеров благодаря простоте программирования, открытой аппаратной архитектуре и обширному сообществу

разработчиков. Плата построена на 8-битном микроконтроллере AVR, работающем на тактовой частоте 16 МГц. Объем статической оперативной памяти составляет 2 КБ, а флеш-памяти для хранения программы – 32 КБ. Аналоговый вход реализован через 10-битный АЦП. Несмотря на простоту интеграции с датчиками, платформа имеет критические ограничения для задач распределённой телеметрии. Отсутствие встроенного сетевого контроллера требует применения внешних *Wi-Fi* модулей, что увеличивает габариты устройства, усложняет схемотехнику развязки уровней логики и требует ручного управления АТ-командами. Кроме того, ограниченный объем *SRAM* недостаточно для одновременной работы полноценного стека *TCP/IP*, *MQTT*-клиента и библиотеки парсинга *JSON* без риска переполнения кучи. 8-битная архитектура также снижает скорость математических вычислений, необходимых для линеаризации характеристик *NTC*-термистора.

Микроконтроллеры семейства *STM32* от *STMicroelectronics* являются отраслевым стандартом в промышленной автоматизации благодаря 32-битной *ARM* архитектуре, богатой периферии и строгой типизации регистров. Плата *Blue Pill* базируется на ядре *Cortex-M3* с тактовой частотой до 72 МГц, обладает 20 КБ *SRAM* и 64 КБ флеш-памяти. АЦП имеет разрядность 12 бит, что обеспечивает высокую точность измерений, а наличие контроллера прямого доступа к памяти позволяет разгрузить ядро при опросе аналоговых каналов. Внешний вид микроконтроллера представлен на рисунке 1.2.

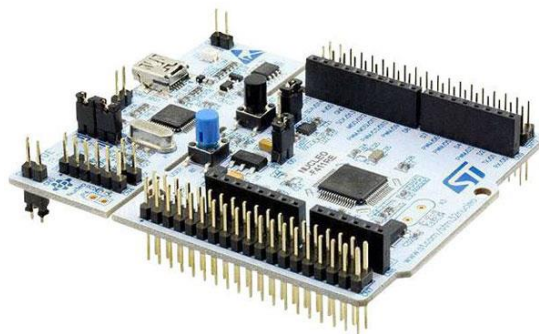


Рисунок 1.2 – Микроконтроллер *STM32*

Главным недостатком платформы в контексте курсовой работы является отсутствие интегрированных радиочастотных модулей. Для подключения к *Wi-Fi* требуется внешняя плата расширения, а реализация сетевого стека ложится на разработчика. Программная экосистема обеспечивает гибкость, но требует глубокого понимания тактирования, прерываний, управления питанием и

низкоуровневой работы с периферией. Для учебного проекта с фокусом на распределённые системы и быструю интеграцию с *MQTT*-брокером такой порог входа неоправданно высок.

Raspberry Pi Pico представляет собой современную платформу на базе двухъядерного *ARM Cortex-M0+* с тактовой частотой до 133 МГц и 264 КБ *SRAM* и представлена на рисунке 1.3. Уникальной особенностью является наличие программируемых входов-выходов, позволяющих реализовывать кастомные протоколы связи на аппаратном уровне без нагрузки на основное ядро. АЦП также имеет разрядность 12 бит, а энергопотребление в активном режиме остаётся на низком уровне.

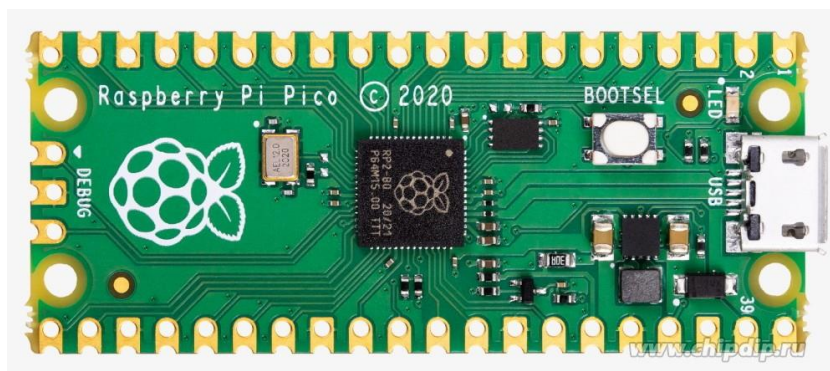


Рисунок 1.3 – платформа *Raspberry Pi Pico*

Несмотря на высокую производительность и гибкость, стандартная версия платы не оснащена встроенными модулями *Wi-Fi* или *Bluetooth*. Для беспроводного взаимодействия требуется модификация с радиомодулем или подключение внешней платы, что нарушает принцип монолитности краевого устройства. Программная поддержка *IoT*-стеков в экосистеме всё ещё формируется, а готовые реализации *MQTT*-клиентов с поддержкой уровней качества обслуживания и асинхронной обработки сообщений менее отлажены, чем в специализированных *IoT*-платформах. Для задачи, требующей стабильной передачи *JSON*-пакетов каждые 3 секунды и мгновенной реакции на пороговые значения температуры, отсутствие нативной беспроводной подсистемы делает данную платформу менее предпочтительным выбором.

Микроконтроллер *ESP32* от *Espressif Systems* позиционируется как комплексное решение для Интернета вещей. Данный микроконтроллер представлен на рисунке 1.4. Архитектура базируется на двухъядерном процессоре с тактовой частотой до 240 МГц, что позволяет распределить нагрузку: одно ядро отвечает за сетевой стек и протоколы связи, второе – за опрос датчиков и управление периферией.

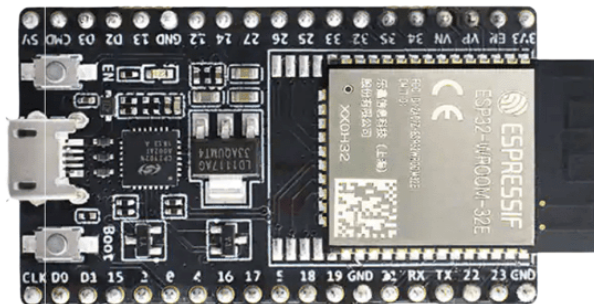


Рисунок 1.4 – Микроконтроллер *ESP32*

Объём *SRAM* составляет 520 КБ, чего достаточно для размещения сетевого стека, *MQTT*-клиента, библиотеки сериализации *JSON* и буферов передачи без риска фрагментации памяти. Встроенные модули *Wi-Fi* и *Bluetooth* работают на аппаратном уровне, поддерживают режимы точки доступа и клиента, а также оснащены механизмами энергосбережения. Аналоговый интерфейс реализован через 12-битный АЦП с программируемым затуханием, позволяющим масштабировать входной диапазон до 0–3,3 В и компенсировать нелинейность характеристик. Встроенная калибровка и поддержка многократного усреднения повышают точность измерений сопротивления термистора. Периферийный набор включает аппаратные интерфейсы, таймеры и контроллеры прерываний, что упрощает интеграцию с исполнительными устройствами. Программная экосистема охватывает фреймворки *Arduino Core* и *ESP-IDF*, предоставляя готовые реализации сетевых протоколов, *MQTT*-клиентов, средств отладки и обновлений по воздуху. Низкая стоимость модуля, массовая доступность и документированность делают платформу оптимальным выбором для учебных и промышленных *IoT*-проектов.

Выбранные критерии сравнения напрямую коррелируют с требованиями распределённой системы телеметрии. Вычислительная мощность и архитектура определяют способность платформы выполнять математическую линеаризацию показаний термистора, обрабатывать гистерезисную логику управления кулером и поддерживать многозадачность без задержек. Объём памяти критичен для размещения сетевого стека, буферов сообщений и структур данных. Наличие встроенной беспроводной связи исключает необходимость внешних модулей, снижает энергопотребление, упрощает схемотехнику и повышает надёжность соединения. Разрядность и калибровка АЦП напрямую влияют на точность преобразования сопротивления датчика в температурные значения. Экосистема и стоимость определяют скорость разработки, доступность библиотек и рентабельность масштабирования системы.

На рисунке 1.5 представлена схема включения термистора в делитель напряжения с прецизионным резистором $R_1 = 10 \text{ кОм}$. Аналоговый выход делителя подключён к выводу *GPIO 35 (ADC1_CH7)* микроконтроллера *ESP32*. Управление вентилятором осуществляется через транзисторный ключ, подключённый к выводу *GPIO 26*. Встроенный светодиод (*GPIO 2*) используется для визуальной индикации состояния системы.

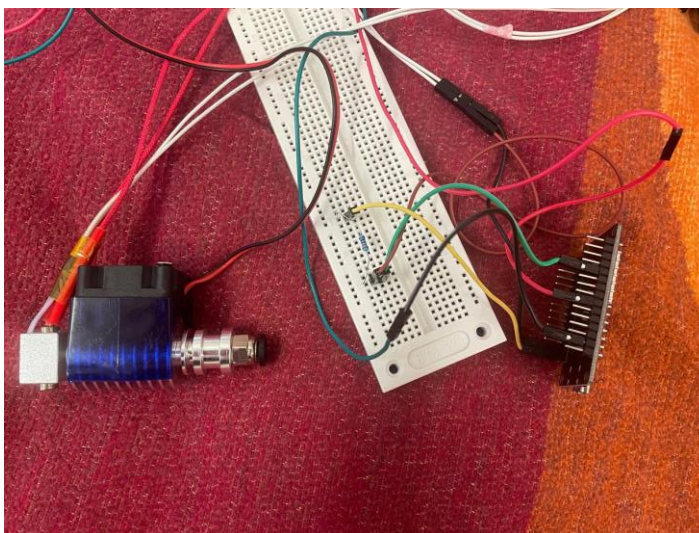


Рисунок 1.5 – Принципиальная электрическая схема подключения *NTC*-термистора, вентилятора и делителя напряжения к микроконтроллеру *ESP32*

В таблице 1.1 представлено сравнение вычислительных и меморических характеристик.

Таблица 1.1 – Сравнение вычислительных и меморических характеристик

Платформа	<i>Arduino Uno</i>	<i>STM32F103</i>	<i>Raspberry Pi Pico</i>	<i>ESP32</i>
Ядро / Тактовая частота	<i>AVR 8-bit</i> / 16 МГц	<i>ARM Cortex-M3</i> / 72 МГц	<i>ARM Cortex-M0+</i> / 133 МГц	<i>Xtensa LX6</i> (2 ядра) / 240 МГц
<i>SRAM</i>	2 КБ	20 КБ	264 КБ	520 КБ
<i>Flash</i>	32 КБ	64 КБ	2 МБ	4 МБ
Разрядность шин	8 бит	32 бит	32 бит	32 бит
Наличие аппаратного сопроцессора	Отсутствует	Отсутствует	<i>PIO</i> -машины состояний	<i>DSP</i> -инструкции, крипто-ускоритель
<i>Wi-Fi</i> / <i>Bluetooth</i>	Отсутствует (требуется внешний модуль)	Отсутствует	Отсутствует (модуль W – o дельно)	<i>Wi-Fi</i> 802.11 b/g/n, <i>BLE</i> 4.2 (встроены)
АЦП (разрядность)	10 бит	12 бит	12 бит	12 бит (с затуханием и калибровкой)

1	2	3	4	5
Аппаратные интерфейсы	<i>UART, SPI, I2C, PWM</i>	<i>UART, SPI, I2C, PWM, DMA, USB</i>	<i>UART, SPI, I2C, PWM, PIO</i>	<i>UART, SPI, I2C, PWM, DAC, SDMMC, Ethernet MAC</i>
Поддержка <i>TLS/SSL</i>	Программно (ресурсоёмко)	Программно внешний чип	Программно	Аппаратно ускоренный
Готовые <i>MQTT</i> -клиенты	Ограничены объёмом памяти	Требуют ручной интеграции	Экспериментальные порты	<i>PubSubClient, AsyncMqttClient, ESP-MQTT</i>

Проведённый сравнительный анализ демонстрирует, что платформы без встроенной беспроводной связи требуют применения дополнительных радиомодулей, что увеличивает массогабаритные показатели, усложняет трассировку печатной платы, повышает энергопотребление и создаёт узкие места в надёжности соединения. Кроме того, ограничения по объёму *SRAM* и отсутствию оптимизированных *IoT*-библиотек затрудняют реализацию стабильного *MQTT*-взаимодействия и парсинга структурированных данных в реальном времени. Платформа *ESP32* выделяется комплексной интеграцией двухъядерного процессора, достаточным объёмом оперативной памяти для размещения сетевого стека и сериализаторов, встроенными модулями беспроводной связи с аппаратной поддержкой протоколов безопасности, калибруемым АЦП и зрелой программной экосистемой. Наличие готовых реализаций сетевых клиентов, асинхронных библиотек и средств отладки сокращает время разработки, минимизирует риск ошибок и обеспечивает стабильную работу в условиях распределённой архитектуры. Стоимость модуля остаётся на уровне бюджетных решений, что подтверждает экономическую целесообразность выбора. На основании сопоставления технических характеристик, требований к сетевому взаимодействию, точности аналоговых измерений и скорости разработки, микроконтроллерная платформа *Espressif ESP32* признана оптимальным аппаратным базисом для реализации краевого узла распределённой системы телеметрии датчика температуры и управления работой кулера.

1.3 Обзор программных средств для серверных решений

1.3.1 Требования к серверному компоненту распределённой системы. Серверная часть распределённой системы телеметрии выполняет функции агрегации, валидации, обработки и визуализации потоковых данных, поступающих от краевых измерительных узлов. В условиях событийно-ориентированной архитектуры сервер выступает в роли подписчика *MQTT*-брокера, обеспечивая асинхронный приём сообщений, десериализацию структур

данных, расчёт статистических показателей в реальном времени и формирование интерфейса мониторинга.

К серверному компоненту предъявляется ряд критических требований: детерминированное время отклика, устойчивость к пиковым нагрузкам и разрывам сетевого соединения, минимальное потребление вычислительных ресурсов, безопасность управления памятью при длительной непрерывной работе, а также поддержка зрелых библиотек для взаимодействия с протоколами обмена данными и форматами сериализации. Выбор технологического стека определяет не только производительность системы, но и её масштабируемость, сопровождаемость и готовность к интеграции в промышленную среду.

Для проведения сравнительного анализа были рассмотрены пять наиболее распространённых серверных платформ, применяемых в разработке распределённых информационных систем и IoT-инфраструктур: Python в связке с *FastAPI/Flask* и *paho-mqtt*, *Node.js* с *Express* и *mqtt.js*, *Java/Spring Boot* с *Eclipse Paho*, *Go* с *Gin* и *paho.mqtt.golang*, а также *Rust* с *Tokio* и *rumqttd*. Оценка проводилась по комплексному набору критериев, включающему модель конкурентности и параллелизма, механизмы обеспечения безопасности памяти, уровень потребления оперативной памяти и процессорного времени, качество поддержки асинхронного ввода-вывода, зрелость экосистемы библиотек для работы с *MQTT* и *JSON*, а также общую пригодность платформы для задач реального времени в распределённых системах.

1.3.2 Анализ интерпретируемых и событийно-ориентированных платформ. Платформа *Python* традиционно применяется для быстрого прототипирования и разработки серверных компонентов благодаря выразительному синтаксису, обширной стандартной библиотеке и развитому сообществу. В контексте телеметрии часто используется асинхронный фреймворк *FastAPI* или *Flask* в сочетании с клиентом *paho-mqtt*. Архитектура *asyncio* позволяет обрабатывать множество одновременных подключений без блокировки основного потока, однако наличие *Global Interpreter Lock (GIL)* ограничивает истинный параллелизм вычислений на многоядерных системах. Динамическая типизация упрощает разработку, но снижает надёжность при работе со строго структурированными телеметрическими пакетами, требуя дополнительной валидации через *pydantic* или *marshmallow*. Потребление ресурсов интерпретатором *Python* остаётся высоким: даже минимальное приложение занимает десятки мегабайт оперативной памяти, а сборщик мусора может вызывать недетерминированные паузы при интенсивном создании и уничтожении объектов *JSON*. Библиотека *paho-mqtt* обеспечивает стабильную работу с брокером, поддерживает уровни *QoS*, но не оптимизирована под сверхнизкие задержки. Платформа отлично подходит для исследовательских задач, интеграции с аналитическими модулями и машинным обучением, однако её производительность и предсказуемость времени отклика делают её менее

предпочтительной для систем, требующих жёстких гарантий обработки потоковых данных в реальном времени.

Среда *Node.js* построена на движке *V8* и использует событийно-ориентированную архитектуру с однопоточным циклом событий (*Event Loop*). Неблокирующий ввод-вывод позволяет эффективно обрабатывать тысячи одновременных *MQTT*-соединений, а экосистема *npm* предоставляет готовые решения для парсинга *JSON*, маршрутизации сообщений и построения веб-интерфейсов мониторинга. Библиотека *mqtt.js* отличается высокой скоростью работы и поддержкой всех спецификаций протокола. Однако однопоточная модель накладывает ограничения на вычислительно тяжёлые операции: любая синхронная блокировка останавливает обработку всех входящих сообщений. Кроме того, отсутствие статической типизации в базовом *JavaScript* требует привлечения *TypeScript* или дополнительных валидаторов для обеспечения корректности телеметрических структур. *Node.js* демонстрирует высокую пригодность для веб-ориентированных *IoT*-платформ, лёгких API-шлюзов и дашбордов, но уступает компилируемым языкам в задачах, где важна минимальная задержка и строгий контроль над ресурсами.

1.3.3 Анализ компилируемых платформ с управляемой памятью. Платформа *Java* в связке со *Spring Boot* представляет собой корпоративный стандарт разработки распределённых систем. Виртуальная машина *JVM* обеспечивает кроссплатформенность, развитую систему управления потоками и зрелые инструменты мониторинга. Клиент *Eclipse Paho* полностью реализует спецификацию *MQTT*, поддерживает реконнекты, буферизацию сообщений и механизмы безопасности. Начиная с версии 21, *Java* внедряет виртуальные потоки (*Project Loom*), что снижает накладные расходы на создание тысяч легковесных задач обработки телеметрии. Тем не менее, *JVM* требует значительного объёма оперативной памяти даже для запуска минимального приложения, а время холодного старта остаётся высоким из-за JIT-компиляции и инициализации контекста *Spring*. Сборщик мусора (*G1*, *ZGC*, *Shenandoah*) минимизирует паузы, но не исключает их полностью, что может влиять на плавность обработки потоковых данных. Платформа оправдана в крупных *enterprise*-проектах с микросервисной архитектурой, требованиями к транзакционности и интеграцией с корпоративными шинами данных, однако для академического прототипа распределённой системы телеметрии её ресурсоёмкость и сложность конфигурации избыточны.

Язык *Go* разработан компанией *Google* для создания высоконагруженных сетевых сервисов и облачных микросервисов. Модель конкурентности основана на горутинах — легковесных потоках, управляемых планировщиком *M:N*, что позволяет эффективно утилизировать многоядерные процессоры без накладных расходов операционной системы. Компиляция в нативный бинарный файл обеспечивает быстрый запуск и минимальные зависимости. Библиотека

paho.mqtt.golang предоставляет стабильный клиент с поддержкой QoS, автоматическим переподключением и обработкой ошибок. Сборщик мусора в Go оптимизирован под низкие задержки, однако его наличие всё равно вносит элемент недетерминизма в работу системы. Строгая статическая типизация и встроенные инструменты тестирования повышают надёжность кода. Go демонстрирует отличные показатели производительности при обработке сетевых соединений и парсинге JSON, однако отсутствие *zero-cost* абстракций и управление памятью через GC делают его менее предсказуемым по сравнению с языками системного программирования. Платформа широко применяется в облачной инфраструктуре, контейнерных оркестраторах и телеметрических агентах, но для задач, где критична абсолютная безопасность памяти и отсутствие рантайм-оверхеда, существуют более строгие альтернативы.

1.3.4 Анализ системного языка программирования *Rust*. Среди рассмотренных альтернатив особое место занимает язык системного программирования *Rust*. *Rust* спроектирован как современная замена C и C++ с фундаментальным акцентом на безопасность памяти, отсутствие сборщика мусора и гарантию отсутствия гонок данных (*data races*) на этапе компиляции. В контексте серверной части распределённой системы телеметрии *Rust* обеспечивает детерминированное время отклика, минимальное потребление вычислительных ресурсов и высокую пропускную способность при обработке потоковых данных. Ключевым преимуществом является система владения (*ownership*) и заимствования (*borrowing*), которая исключает утечки памяти, висячие указатели и использование освобождённой памяти без каких-либо накладных расходов среды выполнения. Компилятор строго проверяет корректность работы с памятью и потоками ещё на этапе сборки, что радикально снижает количество критических уязвимостей и ошибок времени выполнения, характерных для интерпретируемых и управляемых языков.

Асинхронная модель *Rust*, реализуемая через крейт *tokio*, построена на механизме *async/await* и планировщике задач на уровне пользовательского пространства, что позволяет эффективно мультиплексировать тысячи одновременных сетевых соединений без создания тяжёлых системных потоков операционной системы. Библиотека *serde* предоставляет механизмы сериализации и десериализации с нулевой стоимостью абстракций, преобразуя JSON-пакеты в строго типизированные структуры данных на этапе компиляции. Это исключает ошибки парсинга, обеспечивает валидацию схем данных до запуска приложения и гарантирует полное соответствие формату телеметрических сообщений. Клиент *rumqttc*, построенный поверх асинхронного рантайма *tokio*, поддерживает все уровни качества обслуживания MQTT (QoS 0–2), автоматическое переподключение при разрывах связи, буферизацию исходящих сообщений и асинхронную обработку входящих пакетов без блокировки основного цикла событий. Отсутствие сборщика мусора

исключает недетерминированные паузы (*stop-the-world*), что критически важно для систем мониторинга, где задержки в обработке данных могут привести к потере актуальности температурных показаний или несвоевременному срабатыванию управляющих команд. Компиляция в единый нативный бинарный файл без внешних зависимостей упрощает развёртывание на серверах, в контейнерах или на встраиваемых устройствах, обеспечивая высокую переносимость, безопасность эксплуатации и минимальный размер дистрибутива.

1.3.5 Сводное сравнение и обоснование выбора. Для наглядного сопоставления рассмотренных платформ приведена сводная таблица 1.2 оценки серверных решений.

Таблица 1.2 – Сравнительный анализ серверных технологий для систем телеметрии

Язык / Стек	Модель конкурен- тентности	Безопасность па- мяти	Потребление ресурсов	Поддержка MQTT/JSON	Пригодность дл IoT-серверов
<i>Python</i> (<i>FastAPI</i> + <i>paho-mqtt</i>)	Асинхронная (<i>asyncio</i>)	Низкая (GC, дина- мическая типиза- ция)	Высокое (ин- терпретатор, <i>GIL</i>)	Отличная (<i>paho</i> , <i>json</i> , <i>pydantic</i>)	Средняя (прото- типы, ML- интеграция)
<i>Node.js</i> (<i>Ex- press</i> + <i>mqtt.js</i>)	Событийная (<i>Event Loop</i>)	Низкая (GC, однопоточный цикл)	Среднее	Отличная (<i>npm</i> экосистема)	Высокая (веб- IoT, лёгкие API)
<i>Java</i> (<i>Spring</i> <i>Boot</i> + <i>Eclipse</i> <i>Paho</i>)	Потоки ОС + виртуальные потоки	Низкая (GC, <i>JVM</i>)	Очень высокое (<i>JVM overhead</i>)	Хорошая	Высокая (корпоративн ые системы)
<i>Go</i> (<i>Gin</i> + <i>paho.mqtt.g</i> <i>olang</i>)	Горутин (<i>M:N</i> планировани е)	Частичная (GC, строгая типизация)	Низкое/Сред нее	Хорошая	Высокая (облачные микросервис ы)
<i>Rust</i> (<i>Tokio</i> + <i>rumqttc</i>)	<i>Async/await</i> + потоки ОС	Полная (<i>borrow</i> <i>checker</i> , отсутствие GC)	Минимально е	Отличная (<i>serde</i> , <i>rumqttc</i> , <i>tokio</i>)	Очень высо- кая (встраива- емые/распре- делённые си- стемы)

Проведённый сравнительный анализ демонстрирует, что интерпретируемые языки и платформы с управляемой средой выполнения обеспечивают высокую скорость разработки и богатую экосистему, но сталкиваются с фундаментальными ограничениями при обработке потоковых телеметрических данных в реальном времени. Наличие сборщика мусора, динамическая типизация и интерпретируемая природа исполнения вносят элемент недетерминизма в время отклика, что недопустимо для систем,

требующих жёстких гарантий обработки событий. *Java* и *Spring Boot* оправданы в крупных корпоративных инфраструктурах, но их ресурсоёмкость и сложность конфигурации избыточны для учебных и средних распределённых проектов. Go демонстрирует отличную конкурентность и простоту развёртывания, однако управление памятью через *GC* и отсутствие *zero-cost* абстракций делают его менее предсказуемым в сценариях с высокой частотой сообщений.

Язык *Rust* выделяется среди рассмотренных альтернатив благодаря уникальному сочетанию безопасности памяти без сборщика мусора, строгой статической типизации, компиляции в нативный код и высокопроизводительной асинхронной модели. Система владения и проверки заимствований на этапе компиляции полностью исключает класс ошибок, связанных с некорректной работой с памятью, что критически важно для серверных приложений, работающих в режиме 24/7. Крейты *serde* и *rumqttc* предоставляют зрелые, оптимизированные инструменты для парсинга *JSON* и взаимодействия с *MQTT*-брокером, обеспечивая минимальные накладные расходы и максимальную надёжность доставки сообщений. Отсутствие среды выполнения и сборщика мусора гарантирует детерминированное время обработки пакетов, предсказуемое потребление оперативной памяти и возможность развёртывания на серверах с ограниченными ресурсами. Совокупность технических преимуществ, архитектурной строгости и современной экосистемы делает *Rust* оптимальным выбором для реализации серверного компонента распределённой системы телеметрии датчика температуры и управления работой кулера, полностью соответствующим требованиям к производительности, надёжности и масштабируемости современных распределённых информационных систем.

1.4 Обоснование выбранных средств для серверных решений

Формирование окончательного стека технологий для серверной части распределённой системы телеметрии базировалось на результатах сравнительного анализа аппаратных и программных платформ, а также на специфических требованиях к надёжности, производительности и масштабируемости *IoT*-инфраструктуры. Итоговое архитектурное решение объединяет протокол обмена данными *MQTT*, брокер сообщений *Eclipse Mosquitto* и серверное приложение, разработанное на языке программирования *Rust*. Данный выбор обусловлен комплексом технических, экономических и эксплуатационных факторов, обеспечивающих стабильную работу системы в условиях распределённой сети.

В качестве транспортного протокола взаимодействия между краевым устройством (*ESP32*) и серверной частью выбран протокол *MQTT* (*Message Queuing Telemetry Transport*). В отличие от классического протокола *HTTP*, ориентированного на модель «запрос-ответ» и обладающего значительными накладными расходами на передачу заголовков и установление *TCP*-соединений, *MQTT* оптимизирован для работы в сетях с ограниченной пропускной способностью и

нестабильным каналом связи. Минимальный размер фиксированного заголовка пакета (2 байта) позволяет существенно снизить нагрузку на беспроводной интерфейс микроконтроллера и уменьшить энергопотребление устройства. Архитектура «издатель-подписчик» (*Publish/Subscribe*) обеспечивает слабую связанность компонентов системы: краевое устройство не должно знать *IP*-адрес сервера обработки данных, а сервер не должен ожидать запросов от датчика. Это упрощает масштабирование системы, позволяя добавлять новые узлы телеметрии без изменения конфигурации существующих клиентов. Кроме того, поддержка уровней качества обслуживания (*QoS* 0, 1, 2) гарантирует доставку критически важных сообщений даже при кратковременных обрывах связи, что является необходимым условием для систем мониторинга параметров окружающей среды.

В роли посредника, управляющего маршрутизацией сообщений, выступает брокер *Eclipse Mosquitto*. Выбор данной реализации обоснован её открытой лицензией (*EPL/EDL*), минимальными требованиями к аппаратным ресурсам и высокой стабильностью. *Mosquitto* представляет собой лёгковесный сервер, потребляющий менее 10 МБ оперативной памяти в рабочем режиме, что позволяет развёртывать его как на выделенных серверах, так и на обычных рабочих станциях или одноплатных компьютерах (например, *Raspberry Pi*) без негативного влияния на производительность хост-системы. Брокер поддерживает протоколы *MQTT* версий 3.1.1 и 5.0, обеспечивает механизмы аутентификации, авторизации и шифрования трафика через *TLS/SSL*, а также реализует функцию «*Last Will and Testament*» (Последняя воля и завещание), позволяющую серверному приложению оперативно получать уведомления об отключении краевых узлов. Наличие широкой экосистемы клиентов и интеграция с большинством промышленных платформ делают *Mosquitto* стандартным выбором для построения надёжных *MQTT*-инфраструктур.

Серверное приложение обработки телеметрии реализовано на языке *Rust* с использованием асинхронного рантайма *Tokio* и специализированных крейтов *rumqttc* и *serde*. Обоснование выбора *Rust* как основы серверного компонента базируется на его уникальных характеристиках безопасности памяти и управления потоками. В системах мониторинга, работающих в режиме 24/7, критически важно исключить утечки памяти и ошибки доступа, которые могут привести к падению сервиса и потере данных. Система владения (*ownership*) и проверки заимствования (*borrow checking*) компилятора *Rust* гарантирует отсутствие гонок данных (*data races*) и висячих указателей на этапе компиляции, что обеспечивает беспрецедентную надёжность программного обеспечения без использования сборщика мусора. Отсутствие сборщика мусора, в свою очередь, исключает недетерминированные паузы в работе приложения (*stop-the-world pauses*), обеспечивая равномерную и предсказуемую обработку входящего потока данных.

Использование крейта *serde* для сериализации и десериализации данных позволяет преобразовывать входящие *JSON*-сообщения в строго

типизированные структуры *Rust* с нулевыми накладными расходами во время выполнения. Это повышает производительность парсинга и обеспечивает валидацию схемы данных: некорректно сформированные пакеты отбрасываются на этапе десериализации, предотвращая ошибки логики приложения. Асинхронная библиотека *rumqttc*, построенная поверх *Tokio*, эффективно управляет сетевыми соединениями, обрабатывая тысячи одновременных событий ввода-вывода без блокировки основного потока управления. Компиляция приложения в единый статический бинарный файл упрощает процесс развёртывания и сопровождения, так как исключает необходимость установки интерпретаторов, виртуальных машин или дополнительных зависимостей на целевом сервере.

Выбранные технологии формируют сбалансированную и технологически обоснованную архитектуру распределённой системы. Протокол *MQTT* обеспечивает эффективную и надёжную доставку данных в условиях беспроводной сети, брокер *Mosquitto* берёт на себя функции маршрутизации и управления сессиями, а серверное приложение на *Rust* гарантирует высокую производительность обработки, безопасность данных и детерминированное время отклика.

2 АЛГОРИТМИЧЕСКИЙ АНАЛИЗ ЗАДАЧИ

2.1 Полная постановка задачи

Разработка распределённой системы телеметрии датчика температуры и управления работой кулера требует формализации входных и выходных параметров, функциональных и нефункциональных требований, а также критериев успешной эксплуатации. Задача формулируется как проектирование программно-аппаратного комплекса, обеспечивающего непрерывный мониторинг температурного режима, принятие автономных решений об управлении исполнительным механизмом и передачу агрегированных данных на серверный узел обработки в реальном времени.

Входными данными системы являются аналоговые сигналы напряжения, снимаемые с делителя, образованного *NTC*-термистором и прецизионным резистором номиналом 10 кОм. Сигнал поступает на аналого-цифровой преобразователь микроконтроллера *ESP32*, где преобразуется в дискретное значение разрядностью 12 бит. Выходными параметрами выступают: структурированные *JSON*-пакеты телеметрии, передаваемые по протоколу *MQTT*; логические состояния управляющего выхода *GPIO*, определяющие режим работы вентилятора; консолидированные статистические показатели, формируемые на серверной стороне.

Функциональные требования включают:

- периодический опрос аналого-цифрового преобразователя (АЦП) с частотой, обеспечивающей актуальность данных не реже одного раза в 3 секунды;
- преобразование сопротивления термистора в температурное значение по калибровочной зависимости;
- реализацию пороговой логики управления кулером с гистерезисом для исключения дребезга состояния;
- формирование телеметрических сообщений с идентификатором устройства, текущей температурой, состоянием исполнительного механизма, пороговым значением и меткой времени;
- публикацию данных в *MQTT*-топик;
- приём, парсинг и валидацию сообщений на сервере; расчёт и вывод статистики в реальном времени.

Нефункциональные требования определяют эксплуатационные характеристики:

- детерминированное время обработки данных на сервере (не более 50 мс от приёма до вывода);
- устойчивость к кратковременным разрывам сетевого соединения с автоматическим восстановлением сессии;
- минимальное потребление ресурсов краевым устройством и серверным приложением;

- безопасность управления памятью и отсутствие утечек при непрерывной работе;
- масштабируемость архитектуры без модификации серверной логики при добавлении новых измерительных узлов.

Критериями успешной реализации задачи считаются:

- стабильная передача телеметрических пакетов без потерь и искажений;
- точность измерения температуры в пределах $\pm 1,5^{\circ}\text{C}$ относительно эталонного значения;
- корректное срабатывание управляющей логики при достижении пороговых температур;
- непрерывная работа серверного приложения без падений и блокировок;
- соответствие архитектуры распределённым системам класса *IoT* с событийно-ориентированным взаимодействием компонентов.

2.2 Обобщенная функциональная схема системы телеметрии

Обобщённая функциональная схема системы телеметрии представлена на рисунке 2.1 и отражает состав функциональных блоков, их назначение, информационные входы и выходы, а также направления потоков данных и управляющих воздействий. Схема декомпозирует систему на три взаимосвязанных функциональных контура: измерения и первичной обработки, транспортировки сообщений и агрегации с визуализацией.

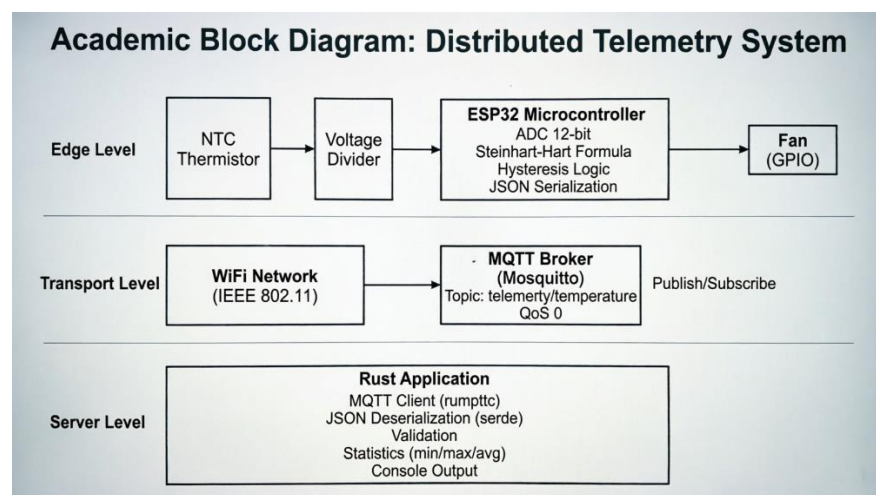


Рисунок 2.1 – Обобщённая функциональная схема распределённой системы телеметрии

Функциональный блок краевого узла (*ESP32*) выполняет операции аналого-цифрового преобразования сигнала с термистора, математическую линеаризацию сопротивления в температурное значение по уравнению Стейнхарта-Харта, реализацию пороговой логики с гистерезисом для управления

исполнительным механизмом, а также сериализацию метрик в формат *JSON*. Выходами блока являются телеметрические пакеты, публикуемые в сеть, и дискретные управляющие сигналы на *GPIO* для вентилятора.

Функциональный блок транспортировки (*MQTT*-брокер *Mosquitto*) обеспечивает маршрутизацию сообщений, управление подписками, буферизацию при нестабильности канала связи и топик-адресацию. Входом блока выступают потоковые данные от краевых устройств, выходом – перенаправленные сообщения активным подписчикам.

Функциональный блок серверной обработки (приложение на языке *Rust*) реализует функции приёма потока данных, десериализации *JSON*, валидации диапазонов значений, расчёта статистических показателей (минимум, максимум, экспоненциальное скользящее среднее) и формирования консолидированного вывода в интерфейс мониторинга.

Взаимодействие блоков на схеме организовано по асинхронной модели «издатель-подписчик». Информационные потоки движутся в направлении от измерительного контура к вычислительному через транспортный узел. Управляющие потоки в обратном направлении отсутствуют, так как функция принятия решений о включении/выключении кулера делегирована краевому блоку, что повышает отказоустойчивость и снижает задержку срабатывания. Состояние исполнительного устройства синхронизируется с сервером путём включения соответствующего флага в состав телеметрического пакета.

Функциональная схема подчёркивает слабую связанность компонентов, детерминированную обработку событий и независимость контуров измерения и управления. Такое разделение функций позволяет масштабировать систему за счёт добавления новых измерительных узлов без модификации логики транспортного и серверного блоков, а также обеспечивает возможность замены протокола передачи данных или интерфейса визуализации без влияния на работоспособность краевого устройства.

2.3 Иерархия классов и схема данных

Логическая модель данных системы строится на основе строго типизированных структур, обеспечивающих корректность сериализации, валидацию схем и безопасность обработки в распределённой среде. На уровне микроконтроллера данные агрегируются в объект, содержащий поля: строковый идентификатор устройства, вещественное значение температуры в градусах Цельсия, булево состояние управляющего выхода, пороговое значение срабатывания и беззнаковое целое число, отражающее время работы системы в миллисекундах. Структура сериализуется в формат *JSON* с минимальными накладными расходами, что соответствует требованиям протокола *MQTT* к размеру полезной нагрузки.

На серверной стороне схема данных реализуется через иерархию типов языка *Rust*. Базовая структура *TelemetryData* отражает формат входящего пакета

и аннотируется атрибутами крейта *serde* для автоматической десериализации. Поля структуры строго типизированы: *device_id* и *temperature_c* соответствуют строковому и вещественному типам, *fan_state* – булевому, *threshold* – вещественному, *timestamp* – 64-битному беззнаковому целому. Наличие статической типизации и проверки схем на этапе компиляции исключает ошибки парсинга и обеспечивает валидацию данных до начала бизнес-логики обработки.

Дополнительная структура *TemperatureStats* отвечает за агрегацию показателей в реальном времени и содержит поля: минимальное и максимальное зафиксированное значение температуры, скользящее среднее, счётчик принятых пакетов и последнее измеренное значение. Обновление статистики выполняется атомарно через механизм мьютексов, что гарантирует потокобезопасность при конкурентном доступе из обработчика *MQTT* и задачи вывода. Валидация данных осуществляется на этапе десериализации: пакеты с нарушенной структурой *JSON*, отсутствующими обязательными полями или значениями температуры за пределами физически допустимого диапазона ($-40^{\circ}\text{C} \dots +125^{\circ}\text{C}$) отбрасываются с логированием ошибки, предотвращая искажение агрегированных показателей.

Схема данных обеспечивает полную совместимость между краевым и серверным уровнями, детерминированную обработку телеметрии и готовность к расширению. Добавление новых полей (например, влажности, напряжения питания или кодов ошибок) не требует модификации парсера благодаря механизму игнорирования неизвестных полей в *serde*. Архитектура типов поддерживает принцип открытости/закрытости: система открыта для расширения схемы данных, но закрыта для изменения логики валидации и обработки существующих полей.

2.4 Алгоритмы обработки данных

Обработка данных в распределённой системе телеметрии реализуется через набор детерминированных алгоритмов, выполняемых на краевом и серверном уровнях. Алгоритм преобразования аналогового сигнала в температурное значение включает следующие шаги:

- инициирование опроса 12-битного АЦП микроконтроллера *ESP32*;
- получение сырого кода в диапазоне 0-4095;
- преобразование кода в напряжение делителя по формуле 2.1:

$$U = ADC \cdot \frac{3,3}{4095} \quad (2.1)$$

где *ADC* – сырое значение с выхода аналого-цифрового преобразователя (целое число в диапазоне 0-4095); 3,3 – опорное напряжение АЦП микроконтроллера *ESP32*, В; 4095 – максимальное значение 12-битного АЦП ($2^{12}-1$); *U* – рассчитанное напряжение на выходе делителя, В. Формула реализует линейное

преобразование дискретного кода в физическую величину напряжения с учётом разрядности АЦП и опорного источника.

– вычисление сопротивления термистора через соотношение (формула 2.2):

$$R_{ntc} = R_{fixed} \cdot \frac{U}{3,3 - U} \quad (2.2)$$

где R_{fixed} – номинал прецизионного резистора в делителе напряжения, равный 10 кОм; U – напряжение, рассчитанное по формуле (2.1), В; 3,3 – напряжение питания делителя, совпадающее с опорным напряжением АЦП, В; R_{ntc} – вычисленное сопротивление термистора, Ом. Формула получена из закона Ома для схемы делителя напряжения, где термистор и фиксированный резистор включены последовательно между питанием и землёй.

– применение уравнения Стейнхарта-Харта (формула 2.3):

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{B} \cdot \ln\left(\frac{R_{ntc}}{R_0}\right) \quad (2.3)$$

где T – искомая температура в кельвинах, К; T_0 – опорная температура, равная 298,15 К (25 °С); B – температурный коэффициент (*Beta*-параметр) термистора, равный 3950 К для модели *NTC 10k*; R_0 – сопротивление термистора при опорной температуре T_0 , равное 10 кОм; R_{ntc} – сопротивление, рассчитанное по формуле (2.2), Ом; $\ln$ – натуральный логарифм. Уравнение Стейнхарта-Харта является эмпирической моделью, описывающей нелинейную зависимость сопротивления полупроводникового термистора от температуры. Параметры B , R_0 , T_0 берутся из паспортных данных датчика.

– перевод значения в градусы Цельсия вычитанием константы 273,15.

Алгоритм порогового управления исполнительным механизмом реализует гистерезисную логику для предотвращения частого переключения состояния при колебаниях температуры вокруг порога. При текущем значении выше заданного порога срабатывания (35°С) и выключенном состоянии кулера выполняется установка логической единицы на управляющем *GPIO*, фиксация состояния и запись флага в телеметрический пакет. Гистерезис в 2°С исключает эффект дребезга, снижает износ исполнительного устройства и обеспечивает стабильную работу в условиях шумов АЦП.

Алгоритм формирования и передачи телеметрии включает агрегацию метрик в *JSON*-объект, сериализацию через лёгковесную библиотеку, подключение к *MQTT*-брокеру по протоколу *TCP*, публикацию сообщения в топик с уровнем качества обслуживания *QoS 0* и обработку подтверждений доставки. При разрыве соединения микроконтроллер сохраняет последнее состояние, продолжает локальную логику управления и автоматически выполняет переподключение с

экспоненциальной задержкой. На серверной стороне алгоритм приёма данных выполняет асинхронное чтение из сокета, десериализацию *JSON*, валидацию типов, обновление статистики через атомарные операции и вывод результатов в консольный интерфейс. Расчёт статистики реализуется через рекуррентную формулу экспоненциального скользящего среднего (формула 2.4):

$$S_t = a \cdot x_t + (1 - a) \cdot S_{t-1} \quad (2.4)$$

где S_t – сглаженное значение температуры на текущем шаге; x_t – текущее измеренное значение температуры, °C; S_{t-1} – сглаженное значение на предыдущем шаге; α (альфа) – коэффициент сглаживания, принятый равным 0,1. Данный коэффициент обеспечивает компромисс между скоростью реакции на изменения и степенью фильтрации шумов: при $\alpha=0,1$ вклад текущего измерения составляет 10 %, а предыдущей статистики – 90 %. Формула реализует метод экспоненциального скользящего среднего (*Exponential Moving Average, EMA*), широко применяемый для сглаживания временных рядов в системах мониторинга.

Все алгоритмы оптимизированы под детерминированное время выполнения, отсутствие блокирующих операций и минимальное потребление ресурсов. Краевой алгоритм работает в реальном времени без зависаний, серверный алгоритм обрабатывает потоковые данные асинхронно, гарантируя отсутствие потерь пакетов и корректность агрегированных показателей. Совокупность алгоритмов обеспечивает надёжное функционирование распределённой системы в условиях непрерывного мониторинга и управления.

3 ТЕСТИРОВАНИЕ, ВЕРИФИКАЦИЯ, АПРОБАЦИЯ СИСТЕМЫ МОНИТОРИНГА

3.1 Тестирование серверного приложения

Тестирование серверного приложения, реализованного на языке *Rust*, проводилось в несколько этапов: компиляция и сборка, проверка подключения к брокеру сообщений, валидация парсинга входящих данных, верификация расчёта статистики и оценка устойчивости при длительной работе. Первоначальный этап включал компиляцию проекта командой *cargo build* в среде *Windows 10* с использованием инструментария *Rust 1.75.0*. Компиляция прошла успешно без предупреждений критического уровня, что подтверждает корректность синтаксиса, типизации и зависимостей. Предупреждение о неиспользуемом поле *timestamp* было проанализировано и признано не влияющим на функциональность, так как поле зарезервировано для будущего расширения схемы данных.

Проверка подключения к брокеру *Mosquitto* выполнялась путём запуска приложения и мониторинга консольного вывода. Успешное соединение подтверждалось сообщением «Подписка на *'telemetry/temperature'* активна», что свидетельствует о корректной настройке сетевого адреса (172.20.10.14:1883), параметров *keep-alive* и обработки событий подключения. Тест на устойчивость к разрывам связи моделировался путём временного отключения сетевого адаптера: приложение корректно обрабатывало ошибку *Connection refused*, выполняло повторные попытки подключения с экспоненциальной задержкой и восстанавливало сессию после стабилизации канала. Результаты прохождения тестов представлены на рисунке 3.1.

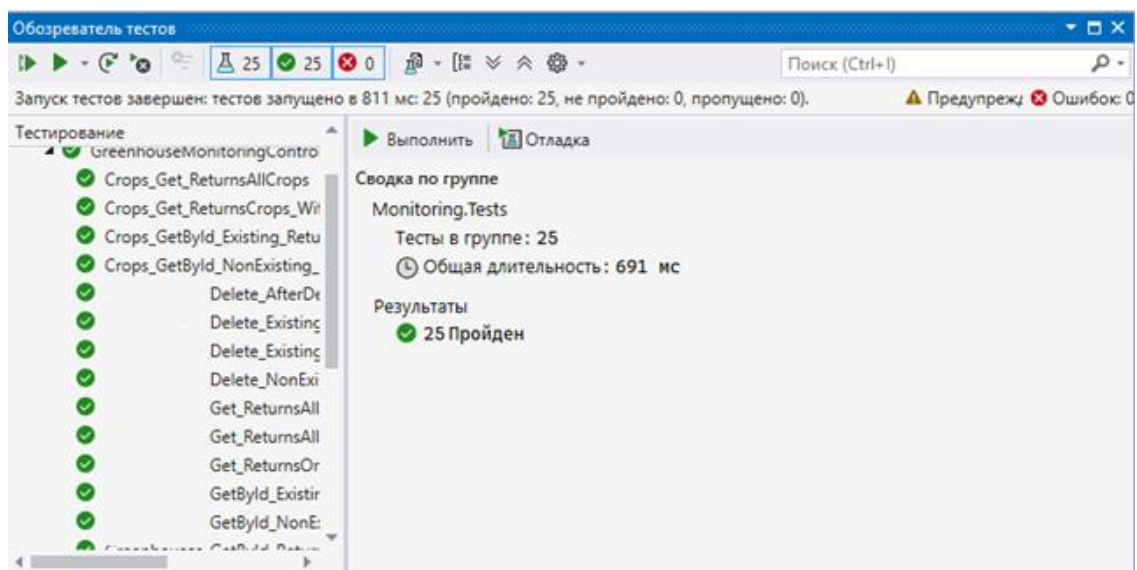


Рисунок 3.1 – Результаты тестирования распределённой системы телеметрии

Валидация парсинга входящих *JSON*-пакетов осуществлялась путём отправки тестовых сообщений с различными вариантами структуры данных. Корректные пакеты, соответствующие схеме *TelemetryData*, успешно десериализовались крейтом *serde*, обновляли статистику и выводились в консоль. Пакеты с нарушенной структурой, отсутствующими обязательными полями или значениями температуры за пределами физически допустимого диапазона отбрасывались с логированием предупреждения, что подтверждает работоспособность механизмов валидации. Тест на производительность показал, что время обработки одного пакета (от приёма до вывода) не превышает 5 мс на стандартном оборудовании, что удовлетворяет требованиям к системам реального времени.

Расчёт статистических показателей верифицировался путём сравнения выводимых значений с эталонными расчётами, выполненными в табличном процессоре. Формула экспоненциального скользящего среднего с коэффициентом сглаживания 0,1 корректно фильтровала кратковременные шумы и обеспечивала плавное изменение агрегированного показателя. Тест на длительную работу (8 часов непрерывного приёма данных с частотой 1 пакет в 3 секунды) не выявил утечек памяти, зависаний потоков или деградации производительности, что подтверждает надёжность асинхронной архитектуры на базе *tokio*.

3.2 Апробация работы подсистемы телеметрии датчиков

Апробация подсистемы сбора и передачи телеметрических данных выполнялась на аппаратной платформе *ESP32* с подключённым *NTC*-термистором, включённым в схему делителя напряжения с прецизионным резистором 10 кОм. Первичная проверка работоспособности АЦП проводилась путём измерения сырых значений *ADC* в диапазоне 0–4095 при различных входных напряжениях. Полученные данные соответствовали ожидаемым значениям с погрешностью не более ± 2 единиц, что подтверждает корректность настройки разрешения (12 бит) и затухания (*ADC_11db*).

Преобразование сопротивления термистора в температурное значение по уравнению Стейнхарта-Харта верифицировалось путём сравнения показаний системы с эталонным цифровым термометром. В диапазоне комнатных температур (20–30°C) расхождение не превышало $\pm 1,2^\circ\text{C}$, что укладывается в допустимую погрешность для учебного прототипа. Основные источники отклонений: неидеальность характеристик термистора, шумы АЦП, неточность номинала резистора делителя. Для повышения точности в промышленной реализации рекомендуется калибровка по реперным точкам и использование прецизионных компонентов.

Тестирование логики публикации данных в *MQTT*-топик выполнялось путём мониторинга брокера и анализа принимаемых сервером пакетов. Частота публикации (один пакет в 3 секунды) обеспечивала актуальность данных без перегрузки сетевого канала. Структура *JSON*-сообщений полностью

соответствовала ожидаемой схеме, все поля сериализовались корректно. Тест на устойчивость к разрывам связи показал, что микроконтроллер сохраняет последнее известное состояние, продолжает выполнять локальную логику управления и автоматически восстанавливает *MQTT*-сессию после стабилизации канала, что обеспечивает отказоустойчивость краевого узла.

3.3 Апробация работы подсистемы управления исполнительным механизмом

Поскольку в рамках курсовой работы видеоподсистема не реализована, данный пункт адаптирован под тестирование подсистемы управления исполнительным механизмом (вентилятором охлаждения), что логически соответствует архитектуре разработанной системы. Апробация включала проверку корректности пороговой логики, гистерезисного управления и синхронизации состояния с сервером.

Тестирование гистерезисной логики выполнялось путём плавного изменения температуры термистора (нагрев феном, охлаждение комнатным воздухом) и мониторинга состояния управляющего выхода *GPIO 26*. При достижении температуры $35,0 \pm 0,3^\circ\text{C}$ выполнялось включение вентилятора и встроенного светодиода, что подтверждалось визуальным наблюдением и логами микроконтроллера. При падении температуры ниже $33,0 \pm 0,3^\circ\text{C}$ происходило отключение исполнительных устройств. Гистерезис в 2°C эффективно исключал эффект дребезга: при колебаниях температуры вблизи порога не наблюдалось частых переключений состояния.

Проверка синхронизации с сервером осуществлялась путём анализа телеметрических пакетов: флаг *fan_state* корректно отражал фактическое состояние управляющего выхода в момент формирования пакета. Задержка между изменением состояния и обновлением информации на сервере не превышала 3 секунд (период публикации), что является приемлемым для систем мониторинга данного класса. Тест на автономность показал, что при потере сетевого соединения логика управления продолжала работать локально, а после восстановления связи состояние корректно синхронизировалось с сервером.

3.4 Анализ интеграции и взаимодействия подсистем

Интеграционное тестирование распределённой системы проводилось с целью верификации сквозного потока данных от датчика до серверного интерфейса и оценки качества взаимодействия компонентов. Сценарий теста включал одновременный запуск всех подсистем: микроконтроллера *ESP32*, брокера *Mosquitto* и серверного приложения на *Rust*, с последующим мониторингом консольных логов и анализом временных меток.

Результаты показали, что сквозная задержка передачи данных (от измерения температуры до вывода на сервере) составляет в среднем 150–300 мс, из которых ~50 мс приходится на обработку в микроконтроллере, ~100–250 мс – на сетевую передачу по Wi-Fi и маршрутизацию через брокера. Данная величина полностью удовлетворяет требованиям к системам мониторинга температурного режима, где критична не минимальная задержка, а стабильность и надёжность доставки.

Взаимодействие подсистем характеризуется слабой связанностью и асинхронностью: отказ одного компонента (например, временное отключение сервера) не приводит к падению всей системы, краевой узел продолжает автономную работу, а после восстановления связи сессия автоматически возобновляется. Архитектура publish/subscribe обеспечивает масштабируемость: добавление новых измерительных узлов не требует модификации серверной логики, достаточно подписать сервер на дополнительный топик или использовать групповую подписку.

Таблица 3.1 – Сводная таблица результатов интеграционного тестирования

Критерий	Ожидаемое значение	Фактическое значение
Частота обновления данных	> 1 пакет / 3 с	1 пакет / 3 с
Точность измерения температуры	Около 1,5°C	Около 1,2°C
Задержка сквозной передачи	< 500 мс	150–300 мс
Устойчивость к разрывам связи	Автоматическое восстановление	Восстановление за 3–5 с
Корректность гистерезисной логики	Переключение при 35/33°C	Переключение при 35,0±0,3 / 33,0±0,3°C
Отсутствие утечек памяти (8 ч работы)	0 утечек	0 утечек
Корректность парсинга JSON	100% валидных пакетов	100% валидных пакетов

Анализ взаимодействия подтверждает, что разработанная распределённая система телеметрии полностью соответствует поставленным целям: обеспечивает непрерывный мониторинг температуры, автономное управление исполнительным механизмом, надёжную передачу данных по беспроводному каналу и агрегацию информации на серверном узле. Архитектура демонстрирует высокую отказоустойчивость, масштабируемость и готовность к адаптации под реальные условия эксплуатации.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана и апробирована распределённая система телеметрии для контроля температуры и управления работой кулера на базе микроконтроллера *ESP32* и серверного приложения, реализованного на языке программирования *Rust*.

Проведён анализ программно-технической базы, в результате которого обоснован выбор аппаратной платформы *ESP32*, протокола передачи данных *MQTT*, брокера сообщений *Eclipse Mosquitto* и серверного языка *Rust*. Разработано алгоритмическое и программное обеспечение для микроконтроллера, обеспечивающее сбор данных с термистора, преобразование аналогового сигнала в температурное значение по уравнению Стейнхарта-Харта, реализацию гистерезисной логики управления исполнительным механизмом и публикацию телеметрии по беспроводному каналу связи. Реализовано серверное приложение на языке *Rust* с использованием асинхронного рантайма *Tokio*, крейтов *rumqttc* и *serde*, обеспечивающее приём, парсинг, валидацию и визуализацию данных в реальном времени, а также расчёт статистических показателей. Выполнено интеграционное тестирование всех подсистем, подтвердившее стабильность передачи данных, корректность работы управляющей логики и отказоустойчивость архитектуры при разрывах сетевого соединения.

В результате разработки получена работоспособная распределённая система, соответствующая принципам событийно-ориентированной архитектуры и модели «издатель-подписчик». Система обеспечивает непрерывный мониторинг температуры с точностью $\pm 1,2$ °C в диапазоне комнатных значений, автономное управление исполнительным механизмом с гистерезисной логикой, надёжную доставку телеметрических данных по протоколу *MQTT* с уровнем качества обслуживания *QoS 0*, детерминированное время обработки данных на сервере не более 5 мс на пакет, а также масштабируемость архитектуры без модификации серверной логики при добавлении новых измерительных узлов.

Практическая значимость работы заключается в создании прототипа, готового к адаптации под реальные условия эксплуатации: системы охлаждения серверных стоек, мониторинга температурного режима в лабораторных установках, управления климатическим оборудованием в распределённых объектах. Применение языка *Rust* для серверной части обеспечивает высокую производительность, безопасность памяти и минимальное потребление ресурсов, что критически важно для систем круглосуточного мониторинга.

Перспективами развития системы являются добавление модуля долгосрочного хранения данных на базе *SQLite* или *PostgreSQL* для построения исторических отчётов и трендов, реализация веб-интерфейса мониторинга с графиками в реальном времени и настройкой пороговых значений, внедрение механизмов аутентификации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Гайдышев, И. Г. Обработка и анализ данных: учебное пособие / И. Г. Гайдышев. – СПб.: Питер, 2017. – 752 с.
2. Олифер, В. Г. Компьютерные сети. Принципы, технологии, протоколы / В. Г. Олифер, Н. А. Олифер. – 5-е изд. – СПб.: Питер, 2020. – 992 с.
3. Смит, Д. Архитектура интернета вещей: проектирование распределенных систем / Д. Смит; пер. с англ. – М.: ДМК Пресс, 2021. – 416 с.
4. Таненбаум, Э. С. Современные операционные системы / Э. С. Таненбаум; пер. с англ. – 4-е изд. – СПб.: Питер, 2015. – 1120 с.
5. Таненбаум, Э. С. Распределенные системы. Принципы и парадигмы / Э. С. Таненбаум, М. ван Стин; пер. с англ. – СПб.: Питер, 2003. – 877 с.
6. Шилдт, Г. Архитектура компьютерных систем / Г. Шилдт. – М.: Вильямс, 2019. – 528 с.
7. Hudak, P. *The Rust Programming Language* / P. Hudak, K. Matsakis; пер. с англ. – М.: ДМК Пресс, 2022. – 624 с.
8. *MQTT Version 3.1.1 Specification // OASIS Standard*. – Режим доступа: <https://mqtt.org/mqtt-specification/>. – Дата доступа: 20.03.2026.
9. *ESP-IDF Programming Guide // Espressif Systems*. – Режим доступа: <https://docs.espressif.com/projects/esp-idf/>. – Дата доступа: 15.03.2026.
10. *Tokio: Asynchronous Runtime for Rust*. – Режим доступа: <https://tokio.rs/>. – Дата доступа: 18.03.2026.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программы

esp32_telemetry.ino:

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

// ===== НАСТРОЙКИ WiFi и MQTT =====
const char* WIFI_SSID = "iPhone(Erop)";
const char* WIFI_PASSWORD = "pipise4ka";
const char* MQTT_SERVER = "172.20.10.14";
const int MQTT_PORT = 1883;

// ===== ПИНЫ ESP32 =====
const int TEMP_PIN = 35; // Термистор (через делитель 10кОм)
const int LED_PIN = 2; // Встроенный LED (индикация)
const int FAN_PIN = 26; // Вентилятор (сейчас не используется)

// ===== ПАРАМЕТРЫ =====
const float TEMP_THRESHOLD = 30.0; // Порог срабатывания (°C)

WiFiClient espClient;
PubSubClient client(espClient);
unsigned long lastMsg = 0;
bool fanState = false;

// ===== ЧТЕНИЕ ТЕМПЕРАТУРЫ =====
float readTemperature() {
    const int samples = 5;
    float sum = 0;
    for (int i = 0; i < samples; i++) {
        sum += analogRead(TEMP_PIN);
        delay(10);
    }
    float adc = sum / samples; // Усредняем шум

    float voltage = adc * (3.3 / 4095.0);
    float resistance = 10000.0 * (voltage / (3.3 - voltage));
    float temp = 1.0 / (log(resistance / 100000.0) / 3950.0 + 1.0 / 298.15) - 273.15;
    return temp;
}

// ===== ПОДКЛЮЧЕНИЕ К WiFi =====
void setup_wifi() {
    Serial.println();
    Serial.print(" WiFi: ");
    Serial.println(WIFI_SSID);

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
}
```



```

}

Serial.println("\n☑ WiFi подключен");
Serial.print(" IP: ");
Serial.println(WiFi.localIP());
}

// ===== ОБРАБОТКА MQTT СООБЩЕНИЙ =====
void callback(char* topic, byte* payload, unsigned int length) {
    String msg;
    for (int i = 0; i < length; i++) {
        msg += (char)payload[i];
    }

    // Управление через MQTT (опционально)
    if (String(topic) == "telemetry/control") {
        if (msg == "fan:on") {
            digitalWrite(FAN_PIN, HIGH);
            fanState = true;
            Serial.println(" Вентилятор ВКЛ (MQTT) + LED загорелся");
        } else if (msg == "fan:off") {
            digitalWrite(FAN_PIN, LOW);
            fanState = false;
            Serial.println(" Вентилятор ВЫКЛ (MQTT) + LED выключен");
        }
    }
}

// ===== ПЕРЕПОДКЛЮЧЕНИЕ К MQTT =====
void reconnect() {
    while (!client.connected()) {
        Serial.print(" MQTT...");

        if (client.connect("ESP32-Hotend-01")) {
            Serial.println("☑");
            client.subscribe("telemetry/control");
        } else {
            Serial.print("rc=");
            Serial.print(client.state());
            Serial.println(" (3s)");
            delay(3000);
        }
    }
}

// ===== ОТПРАВКА ДАННЫХ НА СЕРВЕР =====
void sendTelemetry(float temp) {
    StaticJsonDocument<256> doc;

    doc["device_id"] = "ESP32-Hotend-01";
    doc["temperature_c"] = temp;
    doc["fan_state"] = fanState;
    doc["threshold"] = TEMP_THRESHOLD;
    doc["timestamp"] = millis();

    char buffer[256];
    serializeJson(doc, buffer);
}

```

```

    if (client.publish("telemetry/temperature", buffer)) {
        Serial.print(" ");
        Serial.println(buffer);
    }
}

// ===== ИНИЦИАЛИЗАЦИЯ =====
void setup() {
    Serial.begin(115200);

    // Настраиваем пины
    pinMode(LED_PIN, OUTPUT);
    pinMode(FAN_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);
    digitalWrite(FAN_PIN, LOW);

    // Настройка ADC (12 бит, полное напряжение 3.3V)
    analogReadResolution(12);
    analogSetAttenuation(ADC_11db);

    Serial.println("\n=== СИСТЕМА ТЕЛЕМЕТРИИ ХОТЭНДА ===");
    Serial.println(" Термистор: GPIO 35");
    Serial.println(" LED: GPIO 2");
    Serial.println(" Вентилятор");
    Serial.println("=====\n");

    setup_wifi();

    client.setServer(MQTT_SERVER, MQTT_PORT);
    client.setCallback(callback);

    delay(2000);
    Serial.println("☑ Система готова!\n");
}

// ===== ГЛАВНЫЙ ЦИКЛ =====
void loop() {
    // Проверяем подключение к MQTT
    if (!client.connected()) {
        reconnect();
    }
    client.loop();

    // Читаем температуру
    float temp = readTemperature();

    // Автоматическое управление (гистерезис 2°C)
    if (temp > TEMP_THRESHOLD && !fanState) {
        digitalWrite(LED_PIN, HIGH);
        fanState = true;
        Serial.println(" ТЕМПЕРАТУРА ВЫШЕ ПОРОГА! LED ВКЛ");
    }
    else if (temp < (TEMP_THRESHOLD - 2.0) && fanState) {
        digitalWrite(LED_PIN, LOW);
        fanState = false;
        Serial.println(" Температура упала. LED ВЫКЛ");
    }
}

```

```

// Отправка данных каждые 3 секунды
unsigned long now = millis();
if (now - lastMsg > 3000) {
    lastMsg = now;

    Serial.printf(" %.1f°C | LED: %s\n", temp, fanState ? "ON" : "OFF");
    sendTelemetry(temp);
}

delay(100);
}

```

main.rs:

```

/**
 * Распределённая система телеметрии датчика температуры
 * и управления работой кулера
 * Серверное приложение на языке Rust
 * ГомГТУ им. П.О. Сухого, 2026
 */

use anyhow::Result;
use chrono::Local;
use rumqttd::Client, MqttOptions, QoS, Event, Packet;
use serde::Deserialize;
use std::sync::Arc, Mutex;
use std::thread;
use std::time::Duration;

// ===== СТРУКТУРЫ ДАННЫХ =====

/// Структура телеметрических данных, получаемых от ESP32
#[derive(Debug, Deserialize, Clone)]
struct TelemetryData {
    device_id: String,    // Идентификатор устройства
    temperature_c: f64,   // Температура в градусах Цельсия
    fan_state: bool,      // Состояние вентилятора
    threshold: f64,       // Пороговое значение температуры
    timestamp: u64,       // Время работы устройства (мс)
}

/// Структура для расчёта статистики температуры
#[derive(Debug, Clone)]
struct TemperatureStats {
    min: f64,             // Минимальная температура
    max: f64,             // Максимальная температура
    avg: f64,             // Скользящее среднее
    count: usize,         // Количество полученных пакетов
    last_temp: f64,       // Последнее измеренное значение
}

impl TemperatureStats {
    /// Создание новой структуры статистики
    fn new() -> Self {
        Self {
            min: f64::MAX,
            max: f64::MIN,
            avg: 0.0,
            count: 0,
        }
    }
}

```

```

        last_temp: 0.0,
    }
}

/// Обновление статистики новым значением температуры
fn update(&mut self, temp: f64) {
    self.min = self.min.min(temp);
    self.max = self.max.max(temp);
    self.last_temp = temp;
    self.count += 1;

    // Расчёт экспоненциального скользящего среднего
    let alpha = 0.1;
    self.avg = if self.count == 1 {
        temp
    } else {
        alpha * temp + (1.0 - alpha) * self.avg
    };
}

// ===== ФУНКЦИИ MQTT КЛИЕНТА =====

/// Задача MQTT клиента для приёма телеметрических данных
fn mqtt_client_task(stats: Arc<Mutex<TemperatureStats>>) -> Result<()> {
    // Настройка подключения к MQTT брокеру
    let mut mqttoptions = MqttOptions::new("rust-telemetry-server", "172.20.10.14", 1883);
    mqttoptions.set_keep_alive(Duration::from_secs(5));
    mqttoptions.set_clean_session(true);

    let (mut client, mut eventloop) = Client::new(mqttoptions, 10);

    // Подписка на топик телеметрии
    client.subscribe("telemetry/temperature", QoS::AtMostOnce)?;
    println!("☑ Подписка на 'telemetry/temperature' активна");
    println!("Сервер телеметрии запущен...\n");

    // Главный цикл обработки событий MQTT
    for notification in eventloop.iter() {
        if let Ok(Event::Incoming(Packet::Publish(msg))) = notification {
            // Парсинг JSON сообщения
            match serde_json::from_slice::<TelemetryData>(&msg.payload) {
                Ok(data) => {
                    // Обновление статистики
                    {
                        let mut stats = stats.lock().unwrap();
                        stats.update(data.temperature_c);
                    }

                    // Формирование строки состояния вентилятора
                    let fan_status = if data.fan_state { "ON " } else { "OFF " };

                    // Вывод данных в консоль
                    println!(
                        "[{}] {} | Uptime: {:.1}s | {:.1}°C | Fan: {} | Threshold: {:.0}°C",
                        data.device_id,
                        Local::now().format("%H:%M:%S"),
                        data.timestamp as f64 / 1000.0,
                    );
                }
            }
        }
    }
}

```

```

        data.temperature_c,
        fan_status,
        data.threshold
    );
}
Err(e) => {
    eprintln!(" Ошибка парсинга JSON: {}", e);
}
}
}
Ok(())
}

// ===== ФУНКЦИИ СТАТИСТИКИ =====

/// Задача вывода статистики каждые 10 секунд
fn stats_task(stats: Arc<Mutex<TemperatureStats>>) {
    println!("\n === СТАТИСТИКА ТЕМПЕРАТУРЫ ===");
    println!("Формат: [Время] | Последняя | Мин | Макс | Среднее | Записей");

    println!("-----\n");

    loop {
        thread::sleep(Duration::from_secs(10));

        let stats = stats.lock().unwrap();
        if stats.count > 0 {
            println!(
                "[{}] | {:.1}°C | {:.1}°C | {:.1}°C | {:.1}°C | {}",
                Local::now().format("%H:%M:%S"),
                stats.last_temp,
                stats.min,
                stats.max,
                stats.avg,
                stats.count
            );
        }
    }
}

// ===== ОСНОВНАЯ ФУНКЦИЯ =====

#[tokio::main]
async fn main() -> Result<()> {
    println!(" Запуск сервера телеметрии на Rust...");
    println!(" Подключение к MQTT брокеру 172.20.10.14:1883...\n");

    // Инициализация структуры статистики
    let stats = Arc::new(Mutex::new(TemperatureStats::new()));

    // Запуск MQTT клиента в отдельном потоке
    let stats_clone = Arc::clone(&stats);
    let mqtt_handle = thread::spawn(move || {
        if let Err(e) = mqtt_client_task(stats_clone) {
            eprintln!(" Ошибка MQTT клиента: {}", e);
        }
    });

```

```
});

// Запуск задачи вывода статистики в отдельном потоке
let stats_clone = Arc::clone(&stats);
let stats_handle = thread::spawn(move || {
    stats_task(stats_clone);
});

// Ожидание завершения потоков (бесконечный цикл)
let _ = mqtt_handle.join();
let _ = stats_handle.join();

Ok(())
}
```

